

1주 · 리눅스 & Docker

리눅스 커맨드 · 셸 스크립트 · Docker · Compose (5차시, 20H)

1-2 · 셸 스크립트

1-2 · 셸 스크립트

변수 · 조건문 · 반복문으로 스크립트를 작성하고, 로그 집계와 백업 같은 반복 작업을 자동화합니다.

학습 목표

- 셸 · 변수 · 위치 인자 · 종료 코드의 역할을 설명할 수 있습니다.
- if 조건문과 파일 테스트 연산자로 분기하는 스크립트를 작성할 수 있습니다.
- 명령 치환 \$()으로 명령 실행 결과를 변수에 담을 수 있습니다.
- 로그 집계 · 백업 스크립트를 직접 만들어 실행할 수 있습니다.

시간 배분 (4H)

구분	Chapter	시간
이론	셸 스크립트 기초 문법	1H
실습	스크립트 작성 실습	1.5H
실습	데이터 처리 자동화 스크립트	1H
정리 · 회고	스크립트 3종 동작 점검 · 질의응답	0.5H

참고 사전 준비: 1차시에 만든 ~/data-eng/week1 디렉터리와 access.log(8줄) 파일을 그대로 사용합니다. 없다면 1차시의 "텍스트 처리와 파이프라인" 챕터 1단계 로그 생성 명령을 먼저 실행해 주세요.

오늘 만들 것

어제는 명령을 한 줄씩 손으로 입력했습니다. 오늘은 그 명령들을 파일에 순서대로 적어 두고 한 번에 실행합니다. 이것이 쉘 스크립트이고, 자동화의 출발점입니다.

오늘 하루가 끝나면 손에 스크립트 네 개가 남습니다 — 인사 스크립트, 파일 검사기, 로그 집계기, 백업기. 마지막에는 이들을 묶어 매일 돌릴 수 있는 일일 리포트 스크립트를 만듭니다.

쉘 스크립트 기초 문법 (1H)

변수·인자·조건·반복·종료 코드 — 스크립트를 읽고 쓰는 데 필요한 문법을 한 시간에 훑습니다.

왜 스크립트로 옮겨 적는가

손으로 치는 명령에는 세 가지 문제가 있습니다. 어제 무엇을 쳤는지 기억나지 않고, 다른 사람에게 똑같이 시킬 수 없고, 새벽 3시에는 아무도 칠 수 없습니다. 스크립트는 이 셋을 한 번에 해결합니다.

- 스크립트 = 요리 레시피: 레시피에 순서를 적어 두면 누가 만들어도 같은 요리가 나오듯, 스크립트에 명령 순서를 적어 두면 언제 누가 실행해도 같은 결과가 나옵니다. 데이터 파이프라인의 "재현성"이 여기서 시작됩니다.
- 재현성이 왜 중요한가: 데이터가 이상하면 "어떤 처리를 거쳤는지" 되짚어야 합니다. 손으로 친 명령은 되짚을 수 없지만 스크립트는 파일로 남아 있습니다.
- 그리고 스크립트는 Git에 올라갑니다. 처리 로직의 변경 이력이 남는다는 뜻입니다.

첫 줄의 셔뱅(#!)

스크립트 첫 줄에 적는 `#!/bin/bash` 를 셔뱅(shebang)이라고 합니다. "이 레시피는 bash라는 요리사가 읽어야 한다"는 표시입니다. 어제 배운 셸의 종류가 여기서 쓰입니다.

bash

```
1 #!/bin/bash           # bash 로 실행
2 #!/bin/sh             # 더 기본적인 sh 로 실행 (호환성이 넓지만 기능이 적음)
3 #!/usr/bin/env python3 # 파이썬 스크립트에서 자주 보는 형태
```

TIP 셔뱅이 없어도 bash 파일명 으로는 실행됩니다. 하지만 ./파일명 으로 직접 실행하려면 셔뱅이 있어야 합니다. 어느 요리사를 부를지 시스템이 알 수 없기 때문입니다.

변수 — 등호 양옆에 공백을 넣으면 안 됩니다

bash

```
1 # 올바른 선언 (공백 없음)
2 name="데이터"
3 count=5
4
5 # 틀린 선언 — command not found 가 납니다
6 # name = "데이터"
7
8 # 사용할 때는 앞에 $
9 echo "$name 엔지니어링"
10 echo "총 ${count}개"      # 중괄호는 변수명의 경계를 분명히 할 때
```

주의 변수를 쓸 때는 항상 큰따옴표로 감싸세요 — "\$name" 처럼. 따옴표가 없으면 값에 공백이 있을 때 여러 개로 쪼개집니다. 파일 이름에 공백이 든 경우 이것 때문에 스크립트가 깨지는 일이 정말 흔합니다.

위치 인자 — 실행할 때 넘기는 값

스크립트 이름 뒤에 붙여 주는 값이 순서대로 \$1, \$2, \$3 에 들어옵니다. ./greet.sh 홍길동 이라고 실행하면 \$1 에 홍길동이 담깁니다.

표기	뜻
\$0	스크립트 자기 이름
\$1 \$2 \$3 ...	첫 번째, 두 번째, 세 번째 인자
\$#	넘어온 인자의 개수
\$@	모든 인자를 하나씩 (반복문에서 씀)
\$?	직전 명령의 종료 코드

명령 치환 \$() — 명령 결과를 값으로

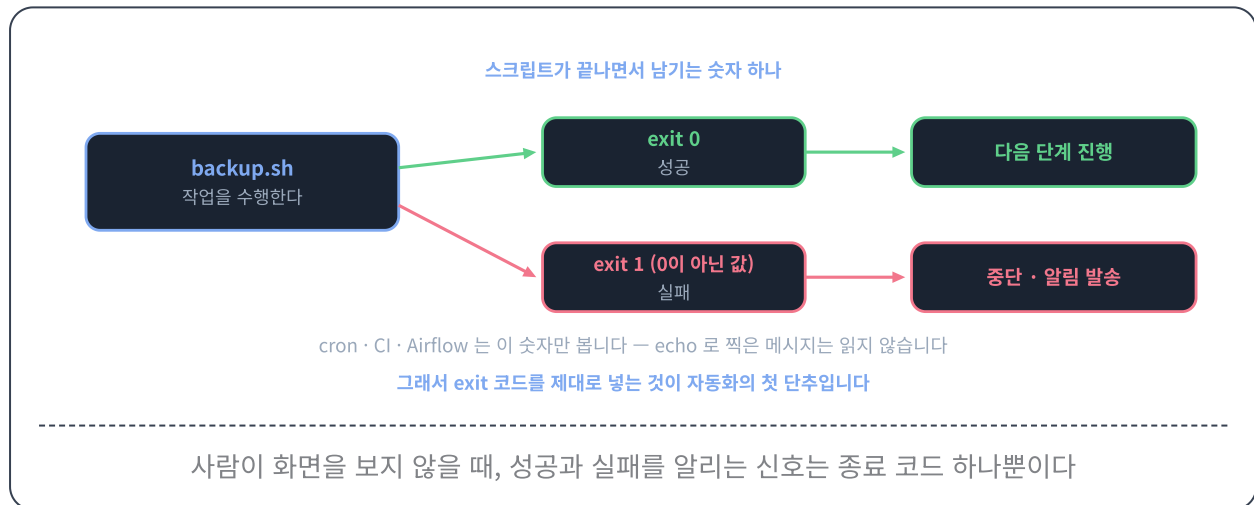
bash

```

1 # 괄호 안 명령을 실행한 결과가 그 자리에 들어옵니다
2 today=$(date +%Y%m%d)
3 echo "오늘은 $today"
4
5 # 파일 줄 수를 변수에 담기
6 lines=$(wc -l < access.log)
7 echo "줄 수: $lines"
8
9 # 백틱 `명령` 도 같은 뜻이지만 옛 문법입니다. $( ) 를 쓰세요 — 중첩이 됩니다.
```

종료 코드 — 자동화가 보는 유일한 신호

스크립트는 끝나면서 숫자 하나를 남깁니다. 0이면 성공, 0이 아니면 실패라는 약속입니다. 사람이 화면을 보고 있을 때는 별로 중요하지 않지만, 새벽에 자동으로 도는 작업에서는 이 숫자가 성패를 판단하는 전부입니다.



bash

```

1 # 직전 명령의 종료 코드 확인
2 ls /etc
3 echo $?          # 0 (성공)
4
5 ls /no-such-dir
6 echo $?          # 2 (실패 — 명령마다 실패 코드는 다를 수 있습니다)
7
8 # 내 스크립트에서 명시적으로 남기기
9 exit 0           # 성공으로 끝냄
10 exit 1          # 실패로 끝냄
```

조건문과 파일 테스트

bash

```
1 if [ -f "access.log" ]; then
2     echo "파일이 있습니다"
3 elif [ -d "logs" ]; then
4     echo "파일은 없지만 logs 폴더는 있습니다"
5 else
6     echo "둘 다 없습니다"
7 fi
```

연산자	참이 되는 조건
-f 경로	그 자리에 일반 파일이 있다
-d 경로	그 자리에 디렉터리가 있다
-e 경로	파일이든 폴더든 존재한다
-s 경로	파일이 있고 크기가 0보다 크다
-z "\$var"	변수가 비어 있다
-n "\$var"	변수에 값이 있다
"\$a" = "\$b"	두 문자열이 같다
\$a -eq \$b	두 숫자가 같다 (-lt -gt -le -ge 도 같은 계열)

주의 [는 문법 기호가 아니라 사실 하나의 명령입니다. 그래서 [뒤와] 앞에 반드시 공백이 있어야 합니다. if [-f "\$f"] 처럼 붙여 쓰면 command not found 가 납니다. 오늘 가장 많이 만나실 오류입니다.

반복문

bash

```
1 # 목록을 하나씩
2 for f in *.log; do
3     echo "처리 중: $f"
4 done
5
6 # 숫자 범위
7 for i in 1 2 3; do
8     echo "$i 번째"
9 done
10
11 # 파일을 한 줄씩 읽기 (실무에서 아주 자주 씁니다)
12 while read -r line; do
13     echo "줄: $line"
14 done < access.log
```

이 1시간에서 꼭 가져갈 것

- 첫 줄 셔뱅이 어느 셸로 읽을지 정한다.
- 변수는 등호 양옆에 공백 없이 선언하고, 쓸 때는 항상 "\$변수" 로 감싼다.
- \$1 은 첫 번째 인자, \$? 는 직전 명령의 종료 코드다.
- [는 명령이므로 대괄호 안쪽에 공백이 필요하다.
- 자동화는 화면 메시지가 아니라 종료 코드를 본다.

스크립트 작성 실습 (1.5H)

인자를 받고, 조건으로 갈라지고, 반복하는 스크립트를 직접 만듭니다. 일부러 한 번 실패시켜 보는 단계가 들어 있습니다.

1단계 — 인자를 받는 첫 스크립트

bash

```
1 cd ~/data-eng/week1
2
3 cat > greet.sh << 'EOF'
4 #!/bin/bash
5 name="$1"
6 echo "안녕하세요, $name 님!"
7 EOF
8
9 chmod +x greet.sh
10 ./greet.sh 데이터엔지니어
```

확인 포인트: "안녕하세요, 데이터엔지니어 님!" 이 출력되면 성공입니다. 인자 없이 ./greet.sh 만 실행하면 이름 자리가 비어서 출력됩니다 — 이것도 한 번 해 보세요.

2단계 — 인자가 없을 때를 막아 줍니다

방금 인자 없이 실행하면 어색한 문장이 나왔습니다. 실무 스크립트는 잘못된 사용법을 안내하고 실패로 끝내야 합니다.

bash

```

1 cat > greet.sh << 'EOF'
2 #!/bin/bash
3
4 # 인자 개수를 확인 - 없으면 사용법을 안내하고 실패로 종료
5 if [ $# -lt 1 ]; then
6     echo "사용법: $0 이름"
7     exit 1
8 fi
9
10 name="$1"
11 echo "안녕하세요, $name 님!"
12 EOF
13
14 ./greet.sh
15 echo $?
16 ./greet.sh 홍길동
17 echo $?
```

확인 포인트: 인자 없이 실행하면 사용법이 나오고 echo \$? 가 1, 이름을 주면 인사말이 나오고 echo \$? 가 0이면 성공입니다.

3단계 — 따옴표의 차이를 눈으로 확인합니다

bash

```

1 # 공백이 든 파일 이름을 만들어 봅니다
2 touch "my report.txt"
3 ls
4
5 f="my report.txt"
6
7 # 따옴표 없이 - 두 개로 쪼개져 실패합니다
8 ls $f
9
10 # 따옴표를 씌우면 하나로 취급됩니다
11 ls "$f"
12
13 # 정리
14 rm "my report.txt"
```

확인 포인트: ls \$f 는 "my: No such file" 처럼 두 번 실패하고, ls "\$f" 는 파일을 정상 출력하면 성공입니다. 이 차이가 실무 스크립트를 깨뜨리는 가장 흔한 원인입니다.

4단계 — 파일 존재를 검사하고 종료 코드를 반환합니다

bash

```
1 cat > check_file.sh << 'EOF'
2 #!/bin/bash
3 target="$1"
4 if [ -f "$target" ]; then
5     echo "$target 파일이 존재합니다."
6 else
7     echo "$target 파일이 없습니다."
8     exit 1
9 fi
10 EOF
11
12 chmod +x check_file.sh
13 ./check_file.sh access.log
14 echo $?
15 ./check_file.sh no-such-file.txt
16 echo $?
```

확인 포인트: access.log 검사 후 echo \$? 는 0, 없는 파일 검사 후에는 1이 출력되면 성공입니다. 이 종료 코드가 자동화 도구들이 성공/실패를 판단하는 기준입니다.

5단계 — 반복문으로 여러 파일을 한 번에

bash

```
1 # 연습용 로그 몇 개 만들기
2 mkdir -p logs
3 echo "ERROR a" > logs/app1.log
4 printf "INFO b\nERROR c\n" > logs/app2.log
5
6 cat > scan_logs.sh << 'EOF'
7 #!/bin/bash
8 dir="${1:-logs}"          # 인자가 없으면 logs 를 기본값으로
9
10 for f in "$dir"/*.log; do
11     lines=$(wc -l < "$f" | tr -d " ")
12     errors=$(grep -c ERROR "$f")
13     echo "$f : 전체 ${lines}줄 · ERROR ${errors}건"
14 done
15 EOF
16
17 chmod +x scan_logs.sh
18 ./scan_logs.sh
```

확인 포인트: logs/app1.log 는 1줄·1건, logs/app2.log 는 2줄·1건으로 두 줄이 출력되면 성공입니다. \${1:-logs} 는 "인자가 없으면 logs 를 쓴다"는 뜻입니다.

6단계 — ShellCheck 로 내 스크립트를 검사합니다

셸 스크립트는 문법 오류가 조용히 넘어가는 일이 많습니다. ShellCheck 는 흔한 실수를 미리 잡아 주는 무료 검사기입니다.

Windows (WSL2)

WSL2 안은 우분투이므로 리눅스와 같습니다. **우분투 터미널에서** 칩니다.

bash

- 1 **sudo apt update**
- 2 **sudo apt install -y shellcheck**

macOS

Homebrew 로 넣습니다. Homebrew 가 없으면 <https://brew.sh> 의 한 줄 명령으로 먼저 설치합니다.

bash

- 1 **brew install shellcheck**

Linux · 우분투

apt 는 시스템을 바꾸므로 **여기서는 sudo** 를 씁니다. (실습 폴더 작업에는 쓰지 않습니다).

bash

- 1 **sudo apt update**
- 2 **sudo apt install -y shellcheck**

설치가 끝나면 **헛다 같은 방향**으로 검사합니다.

bash

```

1 shellcheck scan_logs.sh
2 echo $?          # 0 - 아무것도 안 나오면 통과입니다
3
4 # 지금까지 만든 것을 한꺼번에
5 shellcheck *.sh
6 echo $?          # 지적할 것이 있으면 1

```

확인 포인트: 지금까지 만든 스크립트 가운데 `daily_report.sh`에서만 경고가 잡힙니다. 나머지(`greet` · `check_file` · `count_errors` · `backup` · `scan_logs`)는 통과합니다.

text

```

1 In daily_report.sh line 9:
2 echo "" >> "$out"
3 ^-----^ SC2129 (style): Consider using { cmd1; cmd2; } >>
   file
4                               instead of individual redirects.

```

`>>` 를 여러 번 쓰지 말고 **한 줄이로 묶어 한 번만** 내보내라는 뜻입니다. 파일을 여닫는 횟수가 줄어듭니다. 지금 규모에서는 차이가 없지만, 줄이 수천 개가 되면 달라집니다.

TIP SC2129 는 style 등급입니다 — 틀린 것이 아니라 더 나은 방법이 있다는 뜻입니다. ShellCheck 의 등급은 error · warning · info · style 넷이고, 앞의 둘부터 먼저 고치면 됩니다. 번호(SC2129)로 검색하면 왜 그런지 설명이 나옵니다.

TIP 설치가 어려우면 <https://www.shellcheck.net> 에 코드를 붙여 넣어도 같은 결과를 볼 수 있습니다. 경고 하나하나에 SC숫자 코드가 붙어 있고, 그 번호로 검색하면 왜 문제인지 설명이 나옵니다.

데이터 처리 자동화 스크립트 (1H)

앞에서 만든 조각을 모아 매일 돌릴 수 있는 집계·백업 스크립트를 완성합니다.

1단계 — 로그 집계 스크립트

bash

```
1 cat > count_errors.sh << 'EOF'
2 #!/bin/bash
3 log="$1"
4 if [ ! -f "$log" ]; then
5     echo "로그 파일을 찾을 수 없습니다: $log"
6     exit 1
7 fi
8 total=$(wc -l < "$log" | tr -d " ")
9 errors=$(grep -c ERROR "$log")
10 echo "전체 라인: $total"
11 echo "에러 라인: $errors"
12 EOF
13
14 chmod +x count_errors.sh
15 ./count_errors.sh access.log
```

확인 포인트: "전체 라인: 8", "에러 라인: 3" 이 출력되면 성공입니다(1차시에서 만든 8줄짜리 access.log 기준).

2단계 — 날씨가 붙는 백업 스크립트

bash

```
1 cat > backup.sh << 'EOF'
2 #!/bin/bash
3 src="$1"
4
5 if [ ! -e "$src" ]; then
6     echo "백업할 대상이 없습니다: $src"
7     exit 1
8 fi
9
10 dest="backup_$(date +%Y%m%d)"
11 mkdir -p "$dest"
12 cp -r "$src" "$dest"/
13 echo "$src -> $dest 백업 완료"
14 EOF
15
16 chmod +x backup.sh
17 ./backup.sh logs
18 ls
```

확인 포인트: ls 결과에 backup_오늘날짜 형식의 디렉터리가 생기고, 그 안에 logs 디렉터리가 복사되어 있으면 성공입니다.

3단계 — 조각을 묶어 일일 리포트로

집계와 백업을 순서대로 실행하고, 중간에 하나라도 실패하면 거기서 멈추는 스크립트를 만듭니다. 실제 파이프라인이 이 모양입니다.

bash

```

1 cat > daily_report.sh << 'EOF'
2 #!/bin/bash
3 set -e          # 어느 한 줄이라도 실패하면 즉시 중단
4 set -u          # 선언하지 않은 변수를 쓰면 중단
5
6 log="${1:-access.log}"
7 today=$(date +%Y%m%d)
8 out="report_${today}.txt"
9
10 echo "==== $today 일일 리포트 =====" > "$out"
11 ./count_errors.sh "$log" >> "$out"
12 echo "" >> "$out"
13 echo "[로그 레벨별 집계]" >> "$out"
14 cut -d ' ' -f 1 "$log" | sort | uniq -c | sort -k1,1rn -k2,2 >> "$out"
15
16 ./backup.sh "$log" > /dev/null
17 echo "" >> "$out"
18 echo "백업 완료" >> "$out"
19
20 echo "리포트 생성: $out"
21 EOF
22
23 chmod +x daily_report.sh
24 ./daily_report.sh
25 cat report_*.txt

```

확인 포인트: report_날짜.txt 가 생기고 그 안에 전체/에러 라인 수와 로그 레벨별 집계(INFO 4 · ERROR 3 · WARN 1)가 들어 있으면 성공입니다.

TIP set -e 와 set -u 는 실무 스크립트의 기본 두 줄입니다. set -e 가 없으면 중간에 실패해도 계속 진행해 엉뚱한 결과를 만들고, set -u 가 없으면 변수 이름을 오타 내도 빈 값으로 조용히 돌아갑니다.

4단계 — 자동으로 돌리려면 (맛보기)

스크립트를 정해진 시간에 자동 실행하는 것이 cron 입니다. 이번 주에는 개념만 보고, 실제 스케줄링은 뒤 주차에서 다룹니다.

bash

```
1 # 편집기 열기
2 # crontab -e
3
4 # 형식: 분 시 일 월 요일 실행할명령
5 # 매일 새벽 3시에 실행하려면 아래 한 줄을 넣습니다
6 # 0 3 * * * /home/사용자명/data-eng/week1/daily_report.sh
7
8 # 등록된 목록 확인
9 # crontab -l
```

주의 cron 은 사용자가 로그인했을 때와 환경변수가 다릅니다. 손으로는 되는데 cron 에서만 안 되는 일이 흔한데, 원인은 대부분 경로입니다. cron 에 넣을 때는 명령과 파일 경로를 전부 절대 경로로 쓰세요.

5단계 — 미니 과제

오늘 만든 스크립트를 **고쳐 보는** 과제입니다. 먼저 스스로 해 보시고 정답을 여세요. 세 문제가 이어져 있으니 순서대로 푸시는 것이 좋습니다.

- 1 `count_errors.sh` 를 고쳐, **ERROR** 가 1건 이상이면 `exit 2` 로 끝나게 만드세요. `echo $?` 로 2가 나오는지 확인합니다.

힌트

`errors` 변수에 개수가 들어 있습니다. 숫자 비교는 `-ge` (크거나 같다) 를 씁니다. 조건이 맞을 때만 `exit 2` 아니면 `exit 0`

정답

`[ "$errors" -ge 1 ]` 로 검사하고 `exit 2` 합니다. **종료 코드는 자동화가 보는 유일한 신호**입니다 — 사람이 화면을 보지 않아도 "오류가 있었다" 를 다음 단계에 알릴 수 있습니다. ERROR 가 없는 파일로 돌리면 0 이 나옵니다.

bash

```
1 # count_errors.sh 의 마지막에 추가
2 if [ "$errors" -ge 1 ]; then
3     exit 2
4 fi
5 exit 0
6
7 # 확인
8 ./count_errors.sh access.log ; echo $?      # 2
9 grep -v ERROR access.log > clean.log
10 ./count_errors.sh clean.log ; echo $?      # 0
```

- 2 **1번을 풀고 나면 `daily_report.sh` 가 깨집니다.** 돌려 보고, 왜 그런지와 어떻게 고칠지 설명해 보세요.

힌트

`daily_report.sh` 첫 줄에 `set -e` 가 있습니다. 이것이 무슨 뜻이었는지 떠올려 보세요.

정답

`set -e` 는 어느 한 줄이라도 실패하면 즉시 중단합니다. 그런데 1번에서 `count_errors.sh` 가 이제 `(exit 2 실패)` 로 끝나므로, 리포트가 두 줄만 쓰인 채 멈춥니다. 집계도 백업도 안 됩니다. 종료 코드는 2 입니다. 여기서 "실패"와 "오류가 있음"은 다르다는 것이 요점입니다. ERROR 를 찾는 것은 그 스크립트가 할 일을 제대로 한 것인지 고장이 아닙니다. 그 줄만 `|| true` 로 감싸 중단에서 빼 줍니다.

bash

```
1 # 깨지는 것을 먼저 눈으로 봅니다
2 ./daily_report.sh ; echo $?          # 2
3 cat report_*.txt                     # 두 줄만 있습니다
4
5 # daily_report.sh 에서 그 줄만 고칩니다
6 # 전: ./count_errors.sh "$log" >> "$out"
7 # 후:
8 ./count_errors.sh "$log" >> "$out" || true
```

- 3 `daily_report.sh` 에 「가장 많이 나온 사용자」 를 한 줄 추가하세요. 1-1 에서 만든 조합을 그대로 씁니다.

힌트

`user=` 가 있는 줄만 고르고 → 세 번째 칸을 자르고 → `=` 로 다시 잘라 이름만 → 세고 → 맨 위 한 줄.

정답

1-1 의 파이프라인을 그대로 가져와 결과를 변수에 담고 리포트에 붙입니다. `sort` `rt -k1,1rn -k2,2` 를 쓰는 이유는, 개수가 같을 때 순서가 컴퓨터마다 달라지지 않게 하기 위해서입니다(1-1 에서 다른 함정입니다).

bash

```
1 # daily_report.sh 의 백업 줄 앞에 넣습니다
2 echo "" >> "$out"
3 top_user=$(grep "user=" "$log" | cut -d ' ' -f 3 | cut -
4             d = -f 2 \
5             | sort | uniq -c | sort -k1,1rn -k2,2 | head
6             -n 1)
7 echo "[가장 많이 나온 사용자] $top_user" >> "$out"
8 # 확인 - [가장 많이 나온 사용자] 2 kim
9 ./daily_report.sh && cat report_*.txt
```

- 4 `backup.sh` 를 고쳐, **같은 날 두 번 실행해도 실패하지 않고** `backup_날짜_2` 처럼 번호가 붙게 만드세요.

힌트

폴더가 이미 있으면 `-e` 이름 뒤에 번호를 붙여 다시 봅니다. 없을 때까지 반복하면 됩니다.

정답

`while [ -e "$dest" ]` 로 **번 이름을 찾을 때까지** 번호를 올립니다. `${(n + 1)}` 은 셸의 산술 연산입니다. 세 번 돌리면 `backup_날짜` , `_2` , `_3` 이 생깁니다. 이렇게 **같은 명령을 여러 번 돌려도 탈이 없게** 만드는 것을 **역등(idempotent)** 이라고 하며, 8주차 Airflow 에서 다시 만납니다.

bash

```
1 # backup.sh 의 dest 를 정하는 부분을 바꿉니다
2 base="backup_$(date +%Y%m%d)"
3 dest="$base"
4 n=2
5 while [ -e "$dest" ]; do
6     dest="${base}_${n}"
7     n=$((n + 1))
8 done
9
10 # 확인 - 세 번 돌리면 폴더가 세 개
11 ./backup.sh logs ; ./backup.sh logs ; ./backup.sh logs
12 ls -d backup_*
```

TIP 네 문제가 이어져 있다는 점이 오늘의 진짜 교훈입니다. 스크립트 하나를 고치면 그것을 부르는 다른 스크립트가 함께 흔들립니다. 8주차에 파이프라인을 묶을 때 똑같은 일이 일어나므로, 고친 뒤에는 그것을 쓰는 쪽도 반드시 한 번 돌려 보세요.

정리·회고 (0.5H)

오늘 만든 스크립트 다섯 개가 모두 도는지 확인하고, 문법을 한 장으로 정리합니다.

오늘 만든 스크립트 점검

bash

```
1 cd ~/data-eng/week1
2 ls *.sh
3
4 # 하나씩 돌려 종료 코드까지 확인
5 for s in greet.sh check_file.sh count_errors.sh scan_logs.sh; do
6     echo "--- $s ---"
7     case "$s" in
8         greet.sh)          ./ $s 홍길동 ;;
9         check_file.sh)     ./ $s access.log ;;
10        count_errors.sh)   ./ $s access.log ;;
11        scan_logs.sh)      ./ $s ;;
12    esac
13    echo "종료 코드: $?"
14 done
```

확인 포인트: 네 스크립트가 모두 정상 출력을 내고 종료 코드 0으로 끝나면 오늘 목표를 달성한 것입니다.

문법 요약

하려는 일	문법	주의할 점
셸 지정	<code>#!/bin/bash</code> (첫 줄)	<code>./</code> 로 실행하려면 필수
변수 선언	<code>name="값"</code>	등호 양옆에 공백 금지
변수 사용	<code>"\$name"</code>	항상 큰따옴표로 감싼다
기본값	<code>\${1:-기본값}</code>	인자가 없을 때 대신 쓸 값
인자	<code>\$1 \$2 · \$# · \$@</code>	<code>\$0</code> 은 스크립트 이름
명령 결과 답기	<code>v=\$(명령)</code>	백틱 대신 <code>\$()</code> 를 쓴다
조건	<code>if [-f "\$f"]; then ... fi</code>	대괄호 안쪽에 공백 필수
반복	<code>for f in *.log; do ... done</code>	목록이 비면 패턴 자체가 들어온다
한 줄씩 읽기	<code>while read -r line; do ... done < 파일</code>	<code>-r</code> 을 빼면 백슬래시가 사라진다
종료	<code>exit 0 / exit 1</code>	0 이 성공, 나머지는 실패
안전장치	<code>set -e · set -u</code>	실무 스크립트의 기본 두 줄

오늘의 체크리스트

- 셔백과 `chmod +x`의 역할을 설명할 수 있다.
- 위치 인자 `$1`을 받아 동작하는 `greet.sh`를 만들었고, 인자가 없을 때 사용법을 안내하게 고쳤다.
- 따옴표 유무에 따라 공백 든 파일 이름이 어떻게 달라지는지 직접 확인했다.
- `check_file.sh`가 파일 유무에 따라 종료 코드 0/1을 반환하는 것을 `echo $?` 로 확인했다.
- `for` 반복문으로 여러 로그를 한 번에 집계하는 `scan_logs.sh`를 만들었다.
- 명령 치환 `$()`을 써서 `count_errors.sh`와 `backup.sh`를 완성했다.
- `set -e · set -u` 를 넣은 `daily_report.sh` 로 리포트 파일을 만들었다.

자주 겪는 오류

주의 증상: [: missing 문구나 syntax error가 난다 → 원인: if [-f "\$target"] 에서 대괄호 양옆의 공백이 빠졌습니다. [는 사실 하나의 명령이라 반드시 공백으로 구분해야 합니다. → 해결: [뒤와] 앞에 공백을 넣어 다시 저장합니다.

주의 증상: /bin/bash^M: bad interpreter → 원인: Windows 편집기로 저장해 줄 끝이 CRLF로 되어 있습니다. 리눅스는 LF만 인식합니다. → 해결: VS Code 우측 하단에서 CRLF를 LF로 바꿔 저장하거나, 터미널에서 sed -i "s/\r\$//" 파일명 으로 제거합니다.

주의 증상: 변수 값이 비어 있거나 공백이 든 인자가 쪼개진다 → 원인: 변수를 따옴표 없이 \$target 처럼 썼습니다. 값에 공백이 있으면 여러 개로 갈라집니다. → 해결: 변수 참조는 항상 "\$target" 처럼 큰따옴표로 감싸는 습관을 들입니다.

예상 질문

- Q. bash 와 sh 는 뭐가 다른가요? — sh 가 더 기본적이고 기능이 적습니다. 컨테이너 안에는 bash가 없고 sh만 있는 경우가 있어서, Dockerfile에서 이 차이를 만나게 됩니다(1-4).
- Q. 파이썬으로 짜면 안 되나요? — 됩니다. 다만 파일을 옮기고 명령을 순서대로 부르는 정도의 일은 쉘이 훨씬 짧습니다. 계산이나 자료구조가 들어가기 시작하면 파이썬으로 넘어가는 것이 맞습니다.
- Q. set -e 를 넣었는데도 안 멈추는 경우가 있어요 — 파이프 안에서 실패한 경우입니다. set -o pipefail 을 함께 넣으면 파이프 중간의 실패도 잡힙니다.
- Q. 스크립트를 어디에 두어야 하나요? — 지금은 실습 폴더면 충분합니다. 실무에서는 프로젝트 안의 scripts/ 폴더에 두고 Git으로 함께 관리합니다.

심화 과제 (선택)

- for 반복문으로 logs/ 안의 모든 파일 이름과 라인 수를 한 줄씩 출력하는 스크립트를 작성해 보세요.
- count_errors.sh를 고쳐 ERROR가 1건 이상이면 exit 2로 종료하게 만들고, echo \$? 로 확인해 보세요.
- daily_report.sh 에 set -o pipefail 을 추가하고, 일부러 없는 파일을 넘겨 중간에 멈추는지 확인해 보세요.
- 오늘 만든 스크립트 전부를 shellcheck 로 검사하고, 나온 경고를 하나씩 고쳐 보세요.

참고 링크

- GNU Bash 매뉴얼: <https://www.gnu.org/software/bash/manual/>
- ShellCheck(스크립트 문법 검사기): <https://www.shellcheck.net>

이번 주 산출물

제출은 [산출물 제출](#) 페이지에서 하고, 제출 후 [주차별 산출물 갤러리](#)의 해당 주 섹션에 [1주] 이름 - 산출물명 형식으로 올려 동료와 공유해 주세요.

Built with the LIVE method by 이애본 (Aebon Lee) · DreamIT Biz